

Heap & Priority Queue Practice Problems (Python)

These problems focus only on **Heap** and **Priority Queue** concepts using Python's `heapq` module.

1. Find the Kth Smallest Element

Problem

Given an unsorted array and an integer `k`, find the **kth smallest** element.

Input

```
arr = [7, 10, 4, 3, 20, 15]
k = 3
```

Output

7

Explanation

Sorted array:

```
[3, 4, 7, 10, 15, 20]
```

The 3rd smallest element is **7**.

2. Find the Kth Largest Element

Problem

Given an unsorted array and an integer k , find the **kth largest** element.

Input

```
arr = [3, 2, 1, 5, 6, 4]
k = 2
```

Output

5

Explanation

Sorted in descending order:

```
[6, 5, 4, 3, 2, 1]
```

The 2nd largest element is **5**.

3. Top K Frequent Elements

Problem

Given an integer array and an integer k , return the k most frequently occurring elements.

Input

```
nums = [1, 1, 1, 2, 2, 3]
k = 2
```

Output

```
[1, 2]
```

Explanation

Frequency of each element:

```
1 → 3
2 → 2
3 → 1
```

The two most frequent elements are **1** and **2**.

4. Merge K Sorted Lists

Problem

Given `k` sorted lists, merge them into one sorted list.

Input

```
lists = [
    [1, 4, 5],
    [1, 3, 4],
    [2, 6]
]
```

Output

```
[1, 1, 2, 3, 4, 4, 5, 6]
```

Explanation

Insert the first element from each list into a min heap. Continuously remove the smallest element and insert the next element from the same list until all lists are exhausted.

5. Sort a Nearly Sorted Array

Problem

An array is nearly sorted, meaning every element is at most k positions away from its correct position. Sort the array.

Input

```
arr = [6, 5, 3, 2, 8, 10, 9]
k = 3
```

Output

```
[2, 3, 5, 6, 8, 9, 10]
```

Explanation

Maintain a min heap of size $k + 1$. Repeatedly remove the minimum element and insert the next array element.

6. Last Stone Weight

Problem

You have a collection of stones with different weights.

- Remove the two heaviest stones.
- If both have equal weight, destroy both.
- Otherwise, destroy the lighter stone and insert the difference back into the heap.

Return the weight of the last remaining stone.

Input

```
stones = [2, 7, 4, 1, 8, 1]
```

Output

```
1
```

Explanation

```
8 and 7 → 1
```

```
4 and 2 → 2
```

```
2 and 1 → 1
```

```
1 and 1 → 0
```

```
Remaining stone = 1
```

7. Task Scheduler (Priority Queue)

Problem

Each task has a priority.

Process tasks in the order of **highest priority**, where a smaller priority number indicates a higher priority.

Input

```
tasks = [  
    (3, "Email"),  
    (1, "Bug Fix"),  
    (2, "Meeting")  
]
```

Output

```
Bug Fix  
Meeting  
Email
```

Explanation

The heap always removes the task with the smallest priority number first.

8. Kth Largest Element in a Stream

Problem

Numbers arrive one by one in a stream.

After every insertion, return the **kth largest** element seen so far.

Input

```
k = 3
```

```
Stream:
```

4
5
8
2
3
10

Output

```
Not enough elements
Not enough elements
4
4
4
5
```

Explanation

Maintain a **min heap of size k** containing the largest k elements. The root of the heap is always the k th largest element.

9. Median from Data Stream

Problem

Numbers arrive one by one.

After each insertion, return the current median.

Input

1
2
3

Output

```
1
1.5
2
```

Explanation

Maintain two heaps:

- **Max Heap** → Stores the smaller half of the numbers.
- **Min Heap** → Stores the larger half of the numbers.

Balance both heaps after every insertion to efficiently calculate the median.

10. Find K Pairs with Smallest Sums

Problem

Given two sorted arrays, return the k pairs (u, v) with the smallest sums.

Input

```
nums1 = [1, 7, 11]
nums2 = [2, 4, 6]
k = 3
```

Output

```
[[1, 2], [1, 4], [1, 6]]
```

Explanation

The pair sums are:

$$(1, 2) = 3$$

$$(1, 4) = 5$$

$$(1, 6) = 7$$

$$(7, 2) = 9$$

...

Use a min heap to always retrieve the pair with the smallest sum without generating all possible pairs.